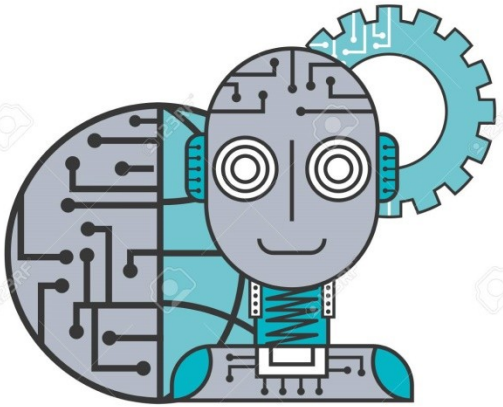
UNIVERSITY OF DERBY

Module 6CM521: Frontiers in AI and Data Science



Ensemble Methods

Ioannis Tsioulis (MC)

TOP 30 GUARDIAN UNIVERSITY GUIDE 2019

TEF Gold Teaching Excellence Framework

derby.ac.uk

Slides acknowledgements:
Dr Aaisha Makkar

Sensitivity:
Internal

Outlook

- Random Forest (RF):
  - Bagging based RF
  - Boosting based RF
- K-NN

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Classification Tree

Disadvantages of the Tree-Structured Approach

Do not predict very accurately compared to other kinds of machine learning models, which is due to the greedy nature of the tree construction algorithms, e.g information gain or Gini index, which introduce overfitting problems very easily.

Trees are unstable

- Small changes to the input data can have large effects on the structure of the tree due to the hierarchical nature of the tree-growing process.

Solution: bagging and boosting.

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Model Averaging

Previously, we focused on machine learning procedures that produce a single set of results:

- A regression equation, with one set of regression coefficients or smoothing parameters.
- A classification regression tree with one set of leaf nodes.

Model selection is often required: a measure of fit associated with each candidate model.

Here, the discussion shifts to machine learning building on many sets of outputs that are **aggregated** to produce results.

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Model Averaging (cont'd)

- The aggregating procedure makes multiple tree growing path over the data.
- On each path, inputs *X* (divided by bagging process) are linked with outputs *Y* just.
- However, of interest now is the collection of all the results from these different trees for aggregating votes.
- Aggregated results have several important benefits.

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Model Averaging (cont'd)

- Averaging over a collection of fitted values can help to avoid overfitting.
- It tends to cancel out the uncommon features of the data captured by a specific model. Therefore, the aggregated results are more stable.
- A large number of fitting attempts can produce very flexible fitting functions.
- Putting the averaging and the flexible fitting functions together has the potential to break the bias-variance tradeoff.

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Model Averaging (cont'd)

- Ideally, one would have two random samples from the same population: a training dataset and a test dataset. The fit measure from the test data would be a better indicator of how accurate the classification is.

- Often there is only a single dataset. The data are split up into several randomly chosen, non-overlapping, partitions of about the same size. With ten partitions, each would be a part of the training data in learning, and serve as the test data in one analysis.

- The model selection is based on choosing the one with smallest cross-validation prediction error.

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Outlook

- Random Forest (RF):
  - Bagging based RF
  - Boosting based RF
- K-NN

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Bagging: Bootstrap Aggregating

There is a very powerful idea in the use of subsamples of the data and in averaging over subsamples through bootstrapping.

Bagging exploits that idea to address the overfitting issue in a more fundamental manner.

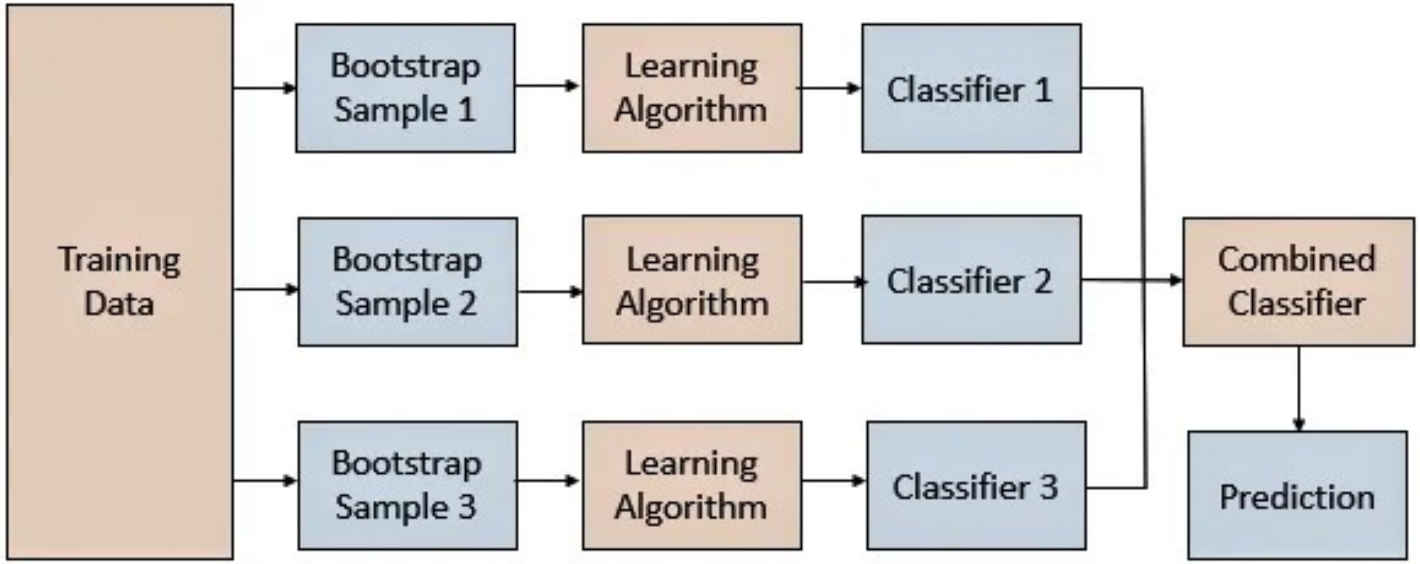
It was invented by Leo Breiman, who called it "**b**ootstrap **agg**regat**ing**" or simply "bagging.“

In a classification tree, bagging takes a majority vote from classifiers trained on bootstrap samples of the training data.

Introduction to bootstrap method:
https://towardsdatascience.com/an-introduction-to-the-bootstrap-method-58bcb51b4d60

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Bagging-based Averaging Learning



UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Example

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns; sns.set()
```

```
from sklearn.datasets import make_blobs

X, y = make_blobs(n_samples=300, centers=4,
                  random_state=0, cluster_std=1.0)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='rainbow');
```



UNIVERSITY OF DERBY

TOP 30 GUARDIAN UNIVERSITY GUIDE 2019

TEF Gold Teaching Excellence Framework

derby.ac.uk

Sensitivity: Internal

UNIVERSITY OF DERBY

Example

```
from sklearn.tree import DecisionTreeClassifier
tree = DecisionTreeClassifier().fit(X, y)
```

Let's write a quick utility function to help us visualize the output of the classifier:

```
def visualize_classifier(model, X, y, ax=None, cmap='rainbow'):
    ax = ax or plt.gca()

    # Plot the training points
    ax.scatter(X[:, 0], X[:, 1], c=y, s=30, cmap=cmap,
               clim=(y.min(), y.max()), zorder=3)
    ax.axis('tight')
    ax.axis('off')
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()

    # fit the estimator
    model.fit(X, y)
    xx, yy = np.meshgrid(np.linspace(*xlim, num=200),
                         np.linspace(*ylim, num=200))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

    # Create a color plot with the results
    n_classes = len(np.unique(y))
    contours = ax.contourf(xx, yy, Z, alpha=0.3,
                           levels=np.arange(n_classes + 1) - 0.5,
                           cmap=cmap, clim=(y.min(), y.max()),
                           zorder=1)

    ax.set(xlim=xlim, ylim=ylim)
```

TOP 30 GUARDIAN UNIVERSITY GUIDE 2019

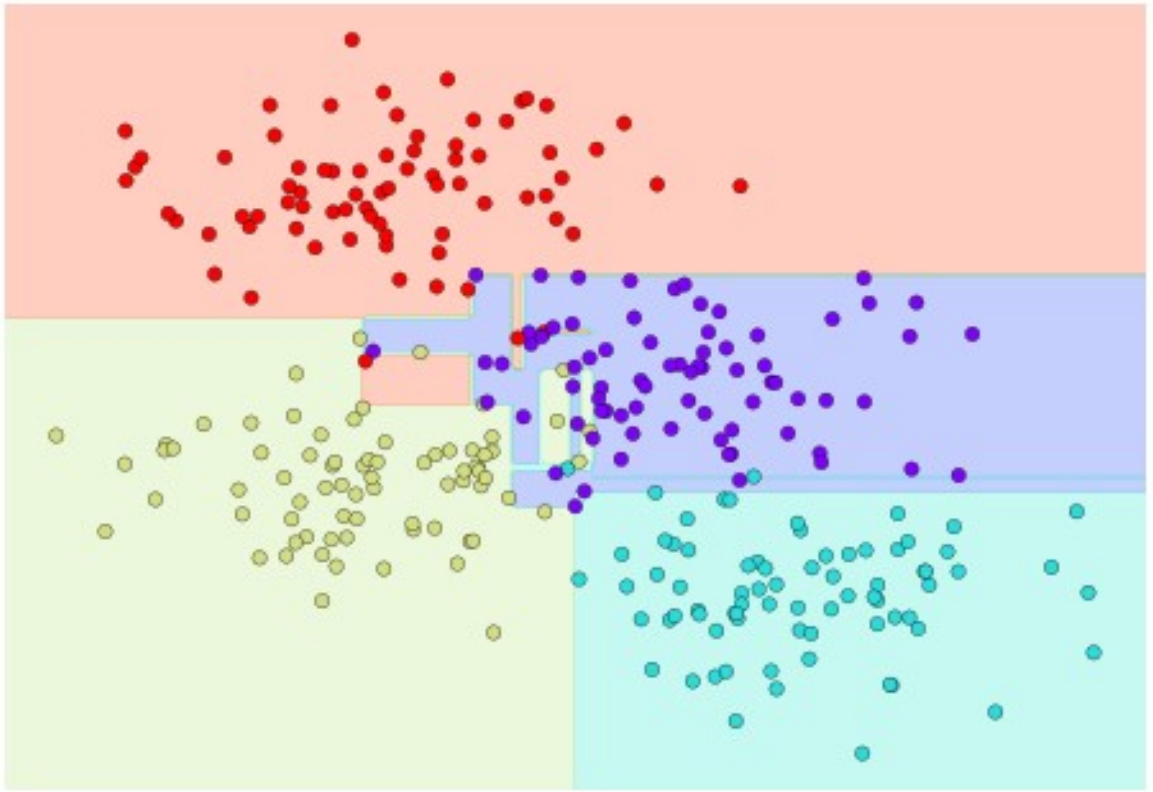
TEF Gold Teaching Excellence Framework

derby.ac.uk

Sensitivity: Internal

Example

```
visualize_classifier(DecisionTreeClassifier(), X, y)
```



UNIVERSITY OF DERBY

TOP 30 GUARDIAN UNIVERSITY GUIDE 2019

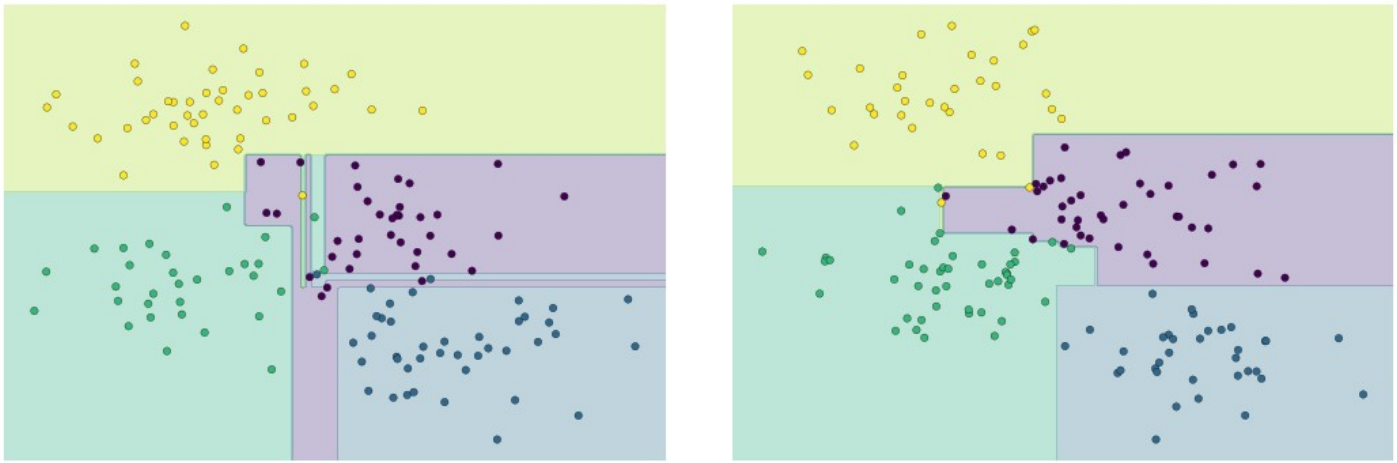
TEF Gold Teaching Excellence Framework

derby.ac.uk

Sensitivity: Internal

Example

Decision trees and over-fitting: another way to see this over-fitting is to look at models trained on different subsets of the data—for example, in this figure we train two different trees, each on half of the original data:



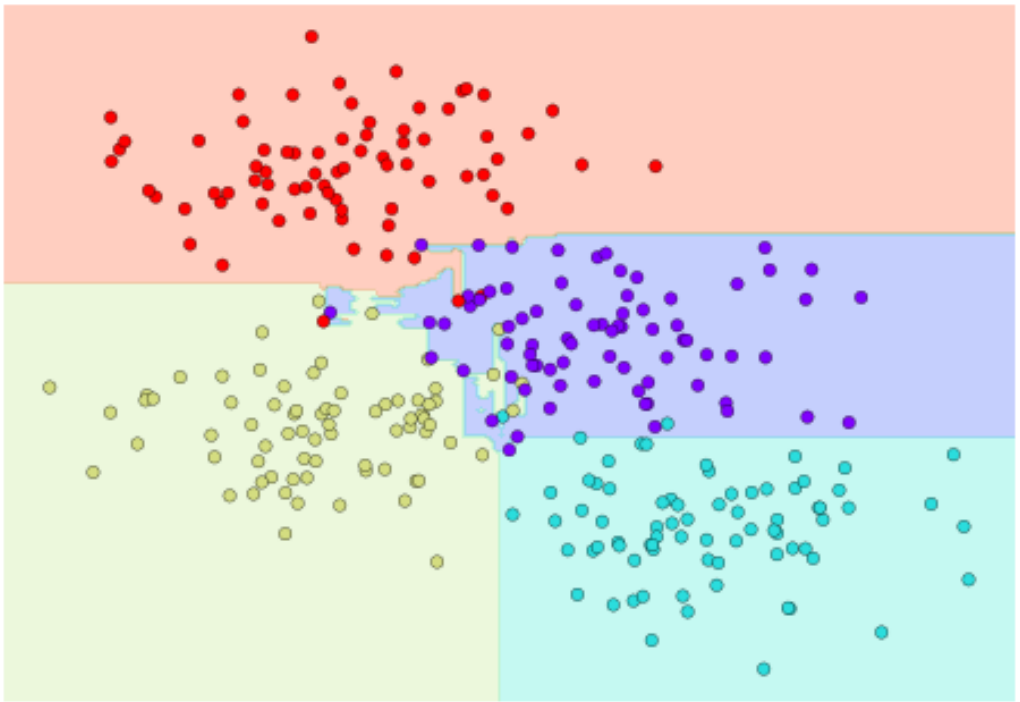
UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Example

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier

tree = DecisionTreeClassifier()
bag = BaggingClassifier(tree, n_estimators=100, max_samples=0.8,
                        random_state=1)

bag.fit(X, y)
visualize_classifier(bag, X, y)
```

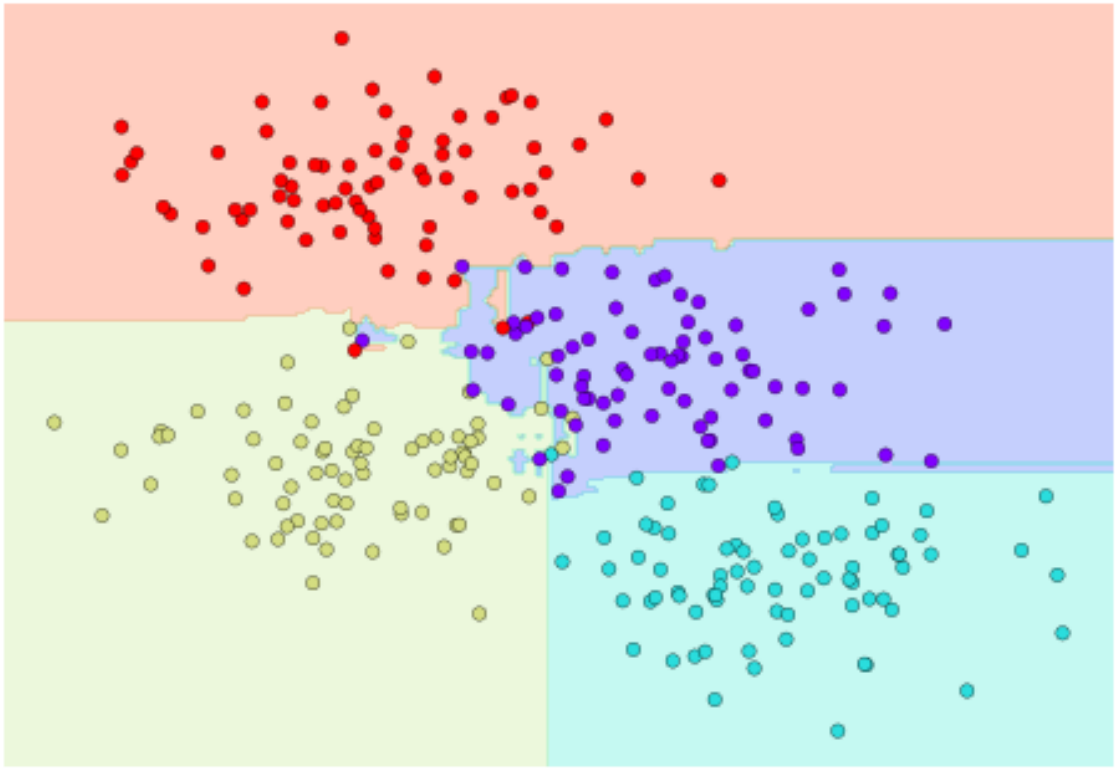


UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Example

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100, random_state=0)
visualize_classifier(model, X, y);
```



UNIVERSITY OF DERBY

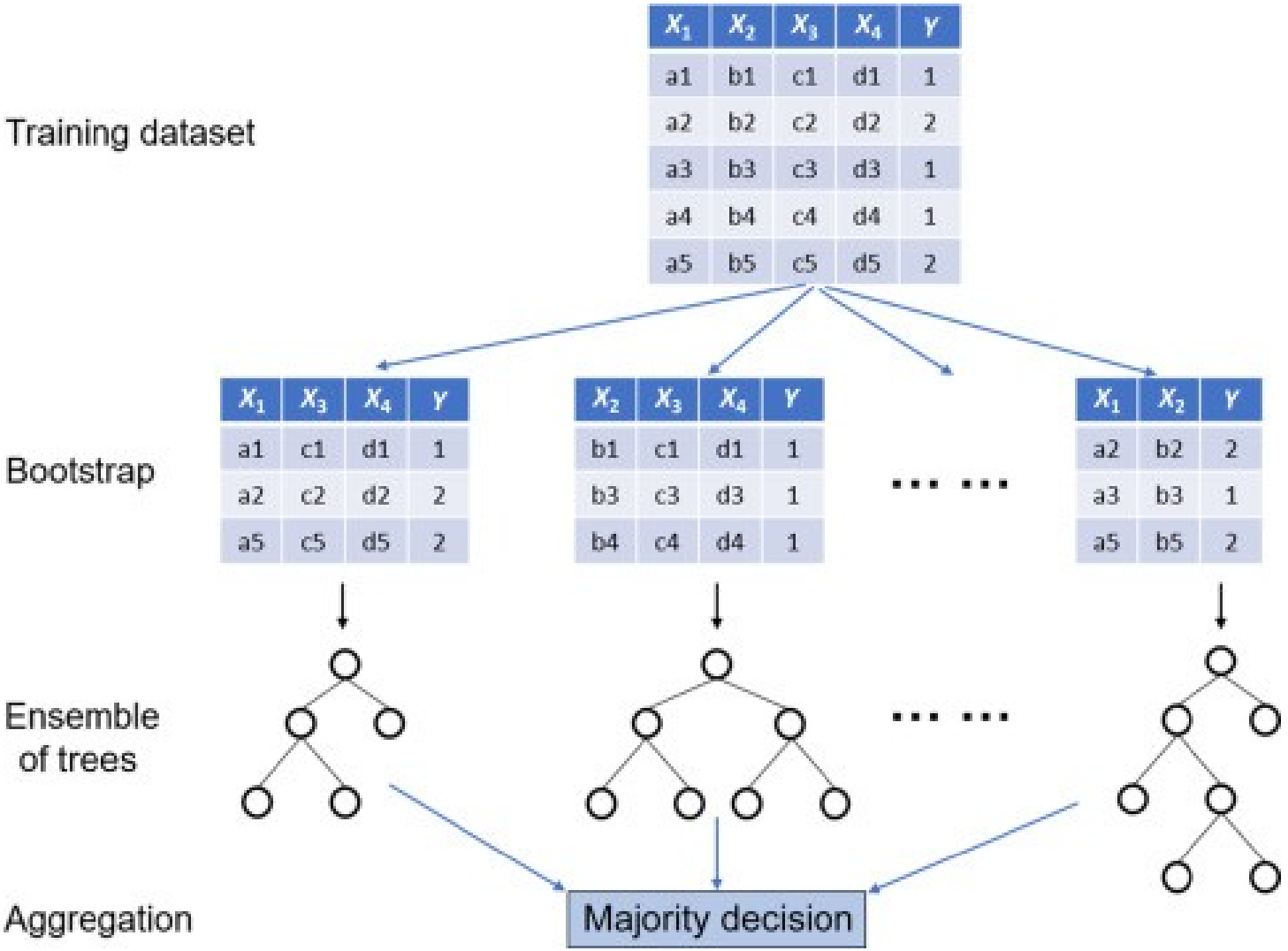
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019

TEF Gold Teaching Excellence Framework

derby.ac.uk

Sensitivity: Internal

Random Forests with bootstrap



X_1	X_2	X_3	X_4	Y
a1	b1	c1	d1	1
a2	b2	c2	d2	2
a3	b3	c3	d3	1
a4	b4	c4	d4	1
a5	b5	c5	d5	2

X_1	X_3	X_4	Y
a1	c1	d1	1
a2	c2	d2	2
a5	c5	d5	2

X_2	X_3	X_4	Y
b1	c1	d1	1
b3	c3	d3	1
b4	c4	d4	1

X_1	X_2	Y
a2	b2	2
a3	b3	1
a5	b5	2

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Sklearn Python

```
from sklearn.ensemble import RandomForestClassifier

# Create the model with 100 trees
model = RandomForestClassifier(n_estimators=100,
                               bootstrap = True,
                               max_features = 'sqrt')
# Fit on training data
model.fit(train, train_labels)
```

Numbers of trees

Default is true means sampling

Here are some commonly used loss functions, gradients, population minimizers, and algorithms to minimize the loss:

Name	Loss	Derivative	f^*	Algorithm
Squared error	$\frac{1}{2}(y_i - f(\mathbf{x}_i))^2$	$y_i - f(\mathbf{x}_i)$	$\mathbb{E}[y\|\mathbf{x}_i]$	L2Boosting
Absolute error	$\|y_i - f(\mathbf{x}_i)\|$	$\text{sgn}(y_i - f(\mathbf{x}_i))$	$\text{median}(y\|\mathbf{x}_i)$	Gradient boosting
Exponential loss	$\exp(-\tilde{y}_i f(\mathbf{x}_i))$	$-\tilde{y}_i \exp(-\tilde{y}_i f(\mathbf{x}_i))$	$\frac{1}{2}\log\frac{\pi_i}{1-\pi_i}$	AdaBoost
Logloss	$\log(1 + e^{-\tilde{y}_i f_i})$	$y_i - \pi_i$	$\frac{1}{2}\log\frac{\pi_i}{1-\pi_i}$	LogitBoost

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Outlook

- Random Forest (RF):
  - Bagging based RF
  - Boosting based RF
- K-NN

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Boosting: Bootstrap Sampling

Boosting, like bagging, is another general approach for improving prediction results for various machine learning methods.

It is also particularly well suited to decision trees.

Rather than resampling the data, however, boosting weights on some examples during learning.

Specifically, in boosting the trees are grown *sequentially*: each tree is grown using information from previously grown trees.

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

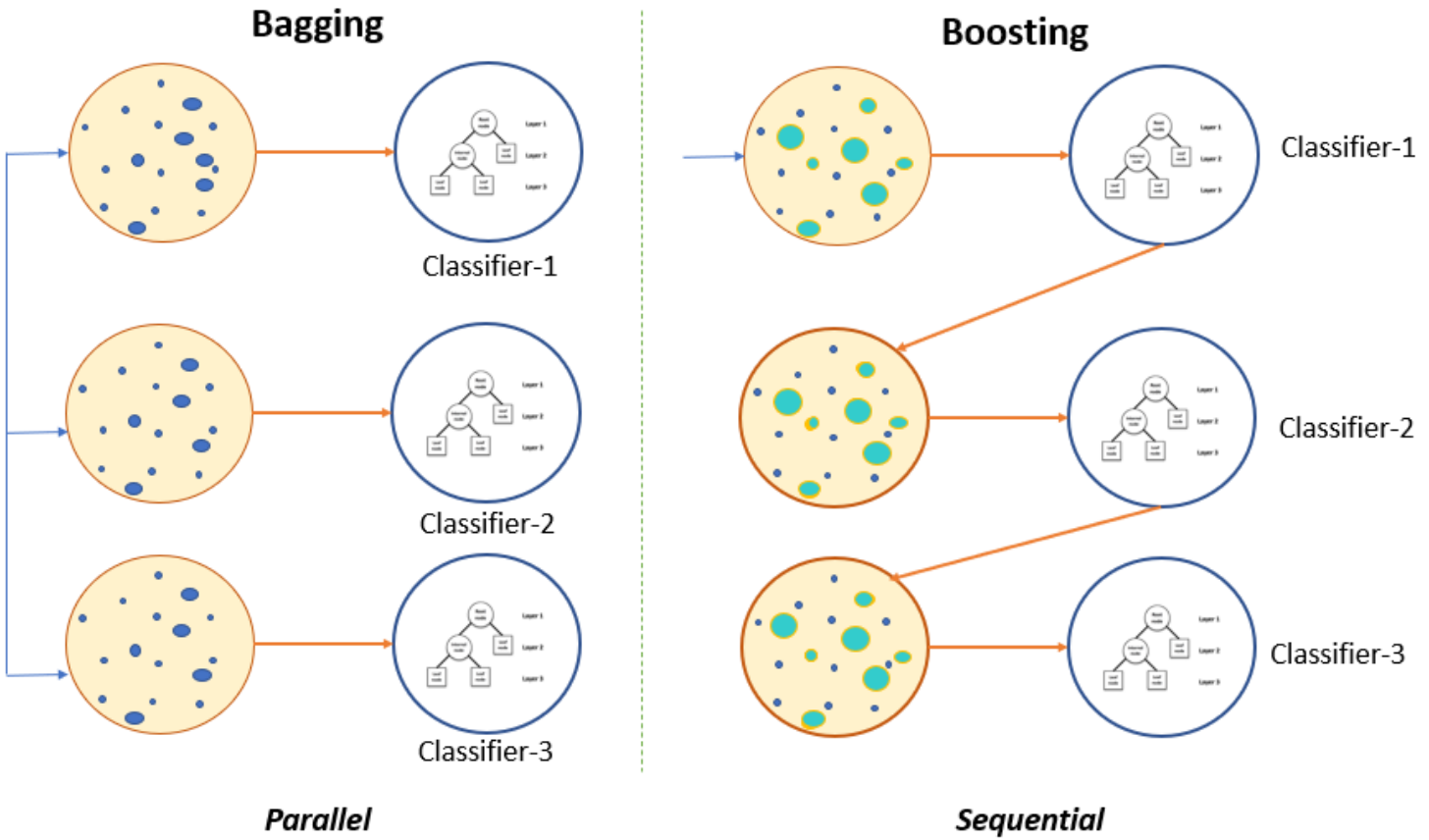
Boosting (cont.)

Here are some commonly used loss functions, gradients, population minimizers, and algorithms to minimize the loss:

Name	Loss	Derivative	f^*	Algorithm
Squared error	$\frac{1}{2}(y_i - f(\mathbf{x}_i))^2$	$y_i - f(\mathbf{x}_i)$	$\mathbb{E}[y\|\mathbf{x}_i]$	L2Boosting
Absolute error	$\|y_i - f(\mathbf{x}_i)\|$	$\text{sgn}(y_i - f(\mathbf{x}_i))$	$\text{median}(y\|\mathbf{x}_i)$	Gradient boosting
Exponential loss	$\exp(-\tilde{y}_i f(\mathbf{x}_i))$	$-\tilde{y}_i \exp(-\tilde{y}_i f(\mathbf{x}_i))$	$\frac{1}{2}\log\frac{\pi_i}{1-\pi_i}$	AdaBoost
Logloss	$\log(1 + e^{-\tilde{y}_i f_i})$	$y_i - \pi_i$	$\frac{1}{2}\log\frac{\pi_i}{1-\pi_i}$	LogitBoost

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Bagging vs Boosting



UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Boosting (cont.)

- Boosting is remarkably resistant to overfitting, and it is fast and simple.
- In fact, it can often continue to improve even when the training error has gone to zero.
- It improves the performance of many kinds of machine learning algorithms.
- Boosting does not work when:
  - Not enough data, base learner is too weak or too strong, and/or susceptible to noisy data.

```
>>> from sklearn.datasets import make_hastie_10_2
>>> from sklearn.ensemble import GradientBoostingClassifier
```

```
>>> X, y = make_hastie_10_2(random_state=0)
>>> X_train, X_test = X[:2000], X[2000:]
>>> y_train, y_test = y[:2000], y[2000:]
```

```
>>> clf = GradientBoostingClassifier(n_estimators=100, learning_rate=1.0,
...     max_depth=1, random_state=0).fit(X_train, y_train)
>>> clf.score(X_test, y_test)
0.913...
```

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Random Forest Regression

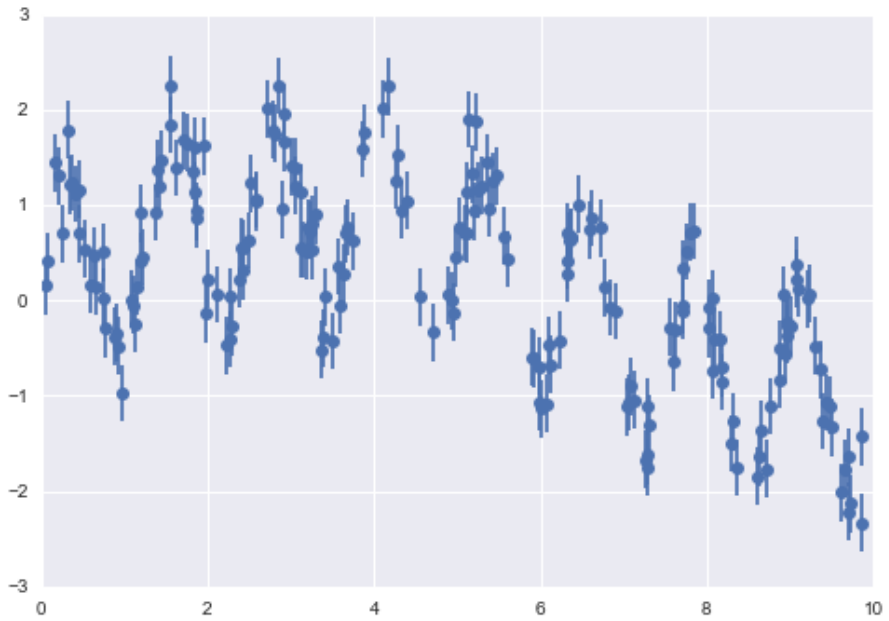
- Non-linear regression problem

```
rng = np.random.RandomState(42)
x = 10 * rng.rand(200)

def model(x, sigma=0.3):
    fast_oscillation = np.sin(5 * x)
    slow_oscillation = np.sin(0.5 * x)
    noise = sigma * rng.randn(len(x))

    return slow_oscillation + fast_oscillation + noise

y = model(x)
plt.errorbar(x, y, 0.3, fmt='o');
```



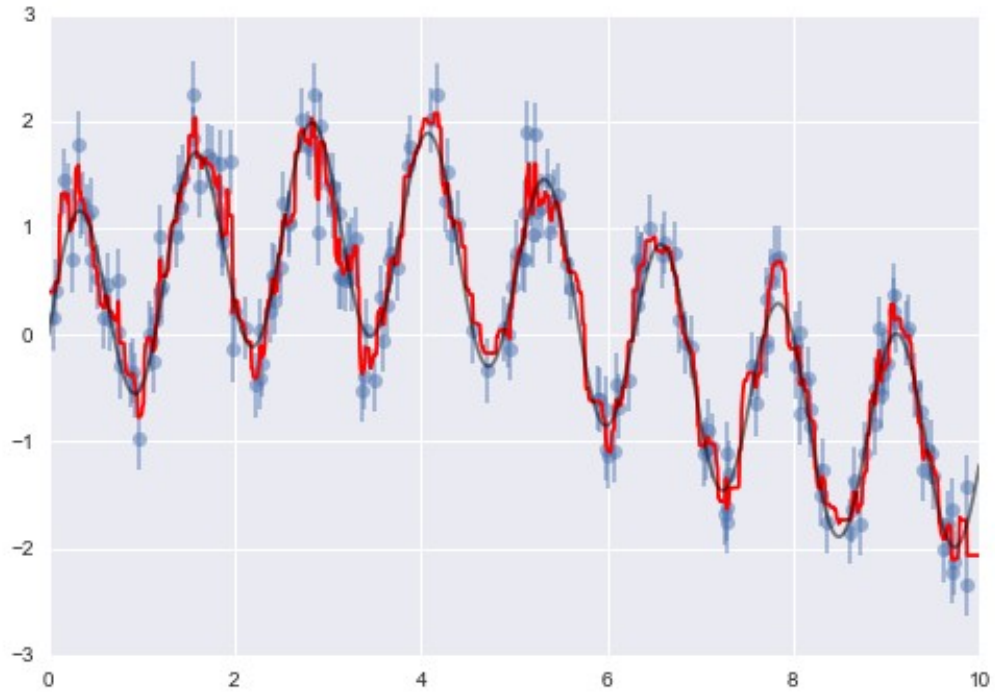
UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Random Forest Regression

```
from sklearn.ensemble import RandomForestRegressor
forest = RandomForestRegressor(200)
forest.fit(x[:, None], y)

xfit = np.linspace(0, 10, 1000)
yfit = forest.predict(xfit[:, None])
ytrue = model(xfit, sigma=0)

plt.errorbar(x, y, 0.3, fmt='o', alpha=0.5)
plt.plot(xfit, yfit, '-r');
plt.plot(xfit, ytrue, '-k', alpha=0.5);
```



UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Outlook

- Random Forest (RF):
  - Bagging based RF
  - Boosting based RF
- K-NN

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

K-NN: An Instance-Based Learning process

Idea:

- Similar examples have similar label.
- Classify new examples like similar training examples.

Algorithm:

- Given some new example x for which we need to predict its class y
- Find most similar training examples
- Classify *x* “like” these most similar examples

Questions:

- How to determine similarity?
- How many similar training examples to consider?
- How to resolve inconsistencies among the training examples?

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

k-NN Approach

The simplest, most used instance-based learning algorithm is the *k*-NN algorithm

k-NN assumes that all instances are points in some *n*-dimensional space and defines neighbors in terms of distance (usually Euclidean in R-space)

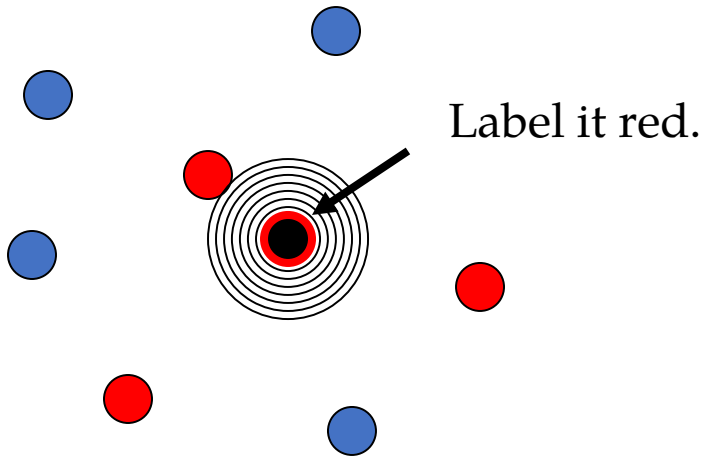
k is the number of neighbors considered

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

1-Nearest Neighbor

One of the simplest of all machine learning classifiers

Simple idea: label a new point the same as the closest known point



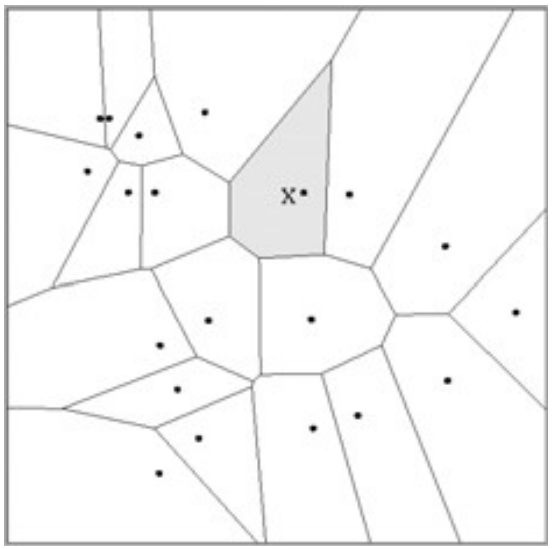
UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

1-Nearest Neighbor

A type of instance-based learning

- Also known as “memory-based” learning

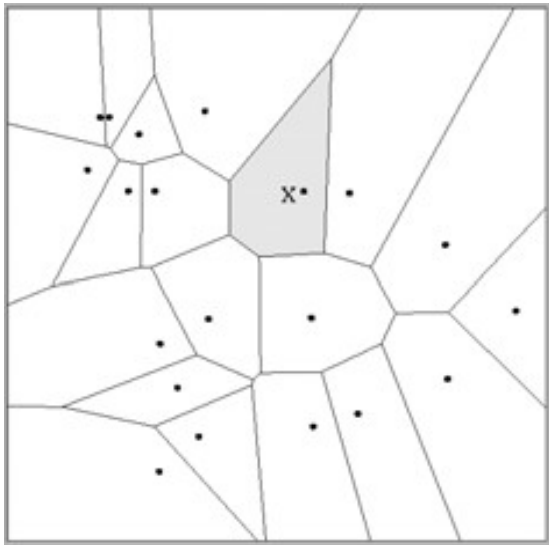
Forms a Voronoi tessellation of the instance space

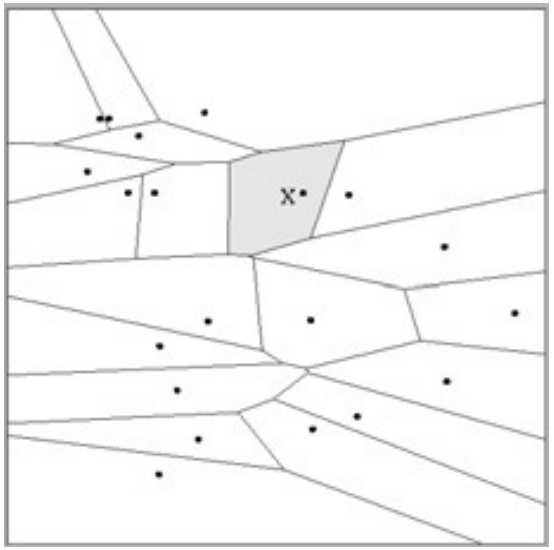


UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Distance Metrics

Different metrics can change the decision surface





$$\text{Dist}(\mathbf{a},\mathbf{b}) = (a_1 - b_1)^2 + (a_2 - b_2)^2$$

$$\text{Dist}(\mathbf{a},\mathbf{b}) = (a_1 - b_1)^2 + (3a_2 - 3b_2)^2$$

Standard Euclidean distance metric:

- Two-dimensional: Dist(a,b) = sqrt($(a_1 - b_1)^2 + (a_2 - b_2)^2$)
- Multivariate: Dist(a,b) = sqrt($\sum (a_i - b_i)^2$)

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

1-NN's Aspects as an Instance-Based Learner:

A distance metric

- Euclidean
- When different units are used for each dimension
  → normalize each dimension by standard deviation
- For discrete data, can use hamming distance
  → D(x1,x2) =number of features on which x1 and x2 differ
- Others (e.g., normal, cosine)

How many nearby neighbors to look at?

- One

How to fit with the local points?

- Just predict the same output as the nearest neighbor.

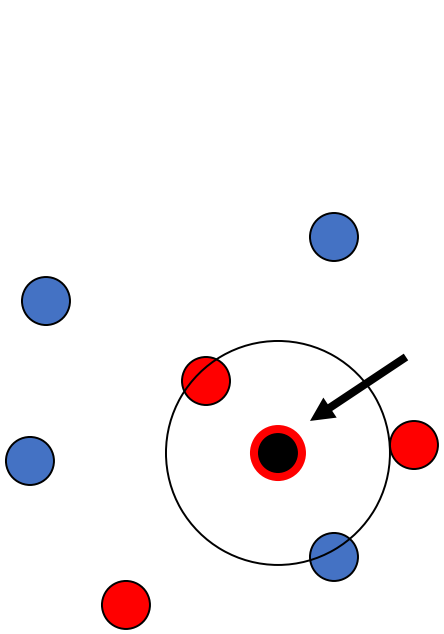
Adapted from "Instance-Based Learning" lecture slides by Andrew Moore, CMU.

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

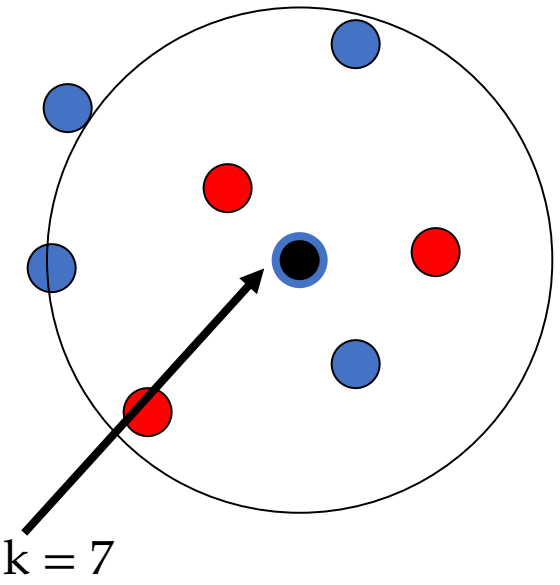
k – Nearest Neighbor

Generalizes 1-NN to smooth away noise in the labels

A new point is now assigned the most frequent label of its *k* nearest neighbors



Label it red, when k = 3



Label it blue, when k = 7

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Selecting the Number of Neighbors

Increase k:

- Makes KNN less sensitive to noise

Decrease k:

- Allows capturing finer structure of space

➔Pick k not too large, but not too small (depends on data)

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Curse-of-Dimensionality

Prediction accuracy can quickly degrade when number of attributes grows.

- Irrelevant attributes easily “swamp” information from relevant attributes
- When many irrelevant attributes, similarity/distance measure becomes less reliable

Remedy

- Try to remove irrelevant attributes in pre-processing step
- Weight attributes differently
- Increase k (but not too much)

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Advantages and Disadvantages of KNN

Need distance/similarity measure and attributes that "match" target function.

For large training sets,

→ Must make a pass through the entire dataset for each classification. This can be prohibitive for large data sets.

Prediction accuracy can quickly degrade when number of attributes grows.

Simple to implement algorithm;

Requires little tuning;

Often performs quite weel!

(Try it first on a new learning problem).

UNIVERSITY OF DERBY

TOP 30 GUARDIAN UNIVERSITY GUIDE 2019

TEF Gold Teaching Excellence Framework

derby.ac.uk

Sensitivity: Internal

K-NN in Python

```
>>> from sklearn.neighbors import KNeighborsClassifier
>>> neigh = KNeighborsClassifier(n_neighbors=3)
>>> neigh.fit(X, y)
```

Parameters:

n_neighbors : *int, default=5*

Number of neighbors to use by default for `kneighbors` queries.

weights : *{'uniform', 'distance'} or callable, default='uniform'*

weight function used in prediction. Possible values:

- 'uniform' : uniform weights. All points in each neighborhood are weighted equally.
- 'distance' : weight points by the inverse of their distance. in this case, closer neighbors of a query point will have a greater influence than neighbors which are further away.
- [callable] : a user-defined function which accepts an array of distances, and returns an array of the same shape containing the weights.

algorithm : *{'auto', 'ball_tree', 'kd_tree', 'brute'}, default='auto'*

Algorithm used to compute the nearest neighbors:

- 'ball_tree' will use `BallTree`
- 'kd_tree' will use `KDTree`
- 'brute' will use a brute-force search.
- 'auto' will attempt to decide the most appropriate algorithm based on the values passed to `fit` method.

https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

How do you decide the number of neighbours in KNN?

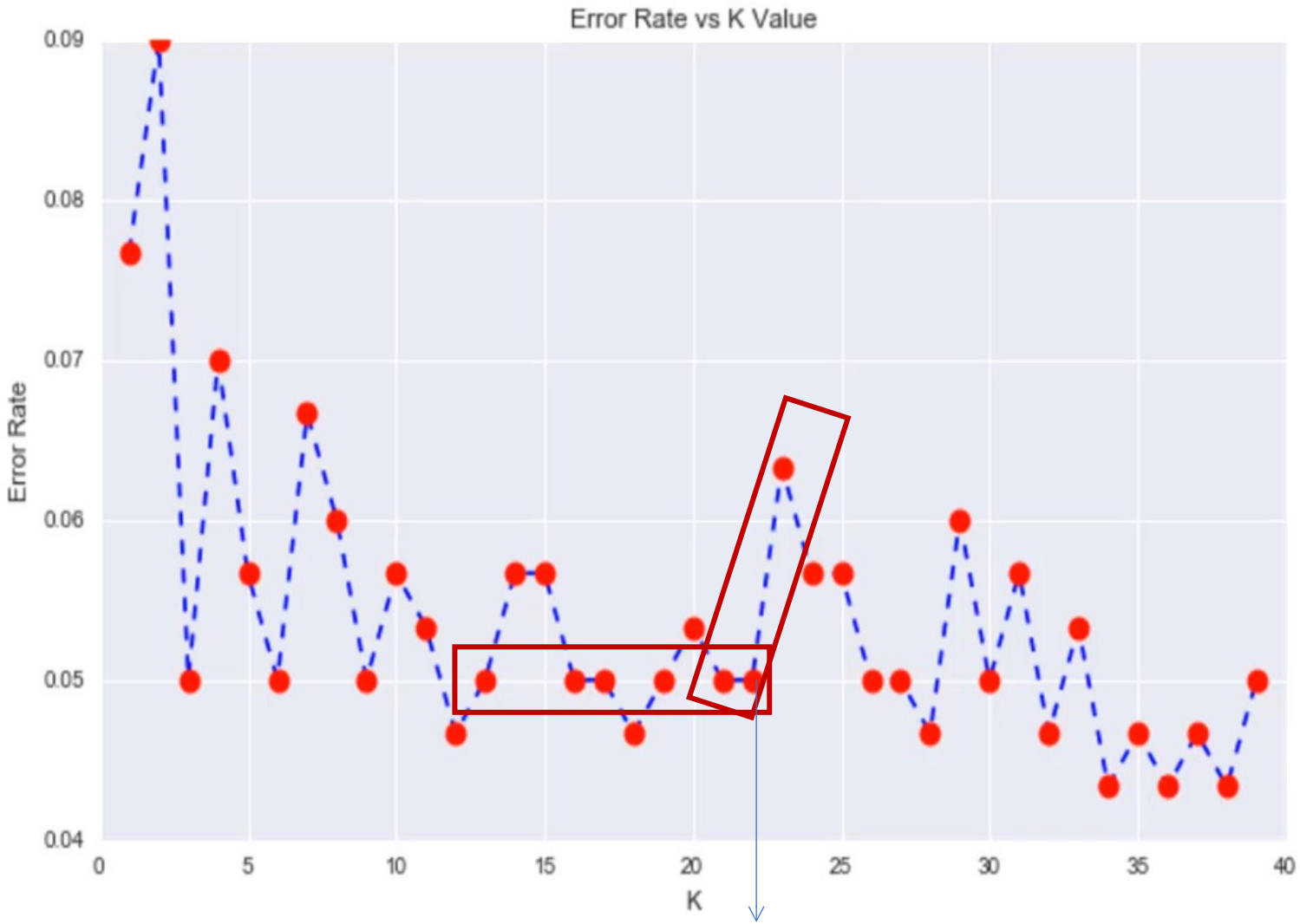
- Research has shown that no optimal number of neighbours suits all kind of data sets. Each dataset has it's own requirements. In the case of a small number of neighbours, the noise will have a higher influence on the result, and a large number of neighbours make it computationally expensive.

- Data scientists choose as an odd number if the number of classes is even

- You can also try Elbow method here.

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Elbow method



22

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Summary

This section contained:

1. A brief introduction to the concept of random forest – an ensemble of randomized decision trees.

2. K-NN algorithm

Lab: Go through a K-NN practice example

Next week:

Unsupervised learning e.g. K-mean to solve clustering problem

UNIVERSITY OF DERBY
TOP 30 GUARDIAN UNIVERSITY GUIDE 2019
TEF Gold Teaching Excellence Framework
derby.ac.uk
Sensitivity: Internal

Online reference

https://www.datacamp.com/community/tutorials/k-nearest-neighbor-classification-scikit-learnn

UNIVERSITY OF DERBY

TOP 30 GUARDIAN UNIVERSITY GUIDE 2019

TEF Gold Teaching Excellence Framework

derby.ac.uk

Sensitivity: Internal